

SecureChat

Implementazione di una chat multiutente tramite socket TCP

Massimiliano Spartà e Simone Adamo

Settembre 2026

1 Introduzione

- Definizione del Problema

2 Metodologie utilizzate

- Risoluzione del problema
- Cifratura del canale con TLS

3 Presentazione dei dati

4 Discussione dei risultati

5 Conclusioni

Il progetto nasce da una consegna precisa: costruire un server di chat centralizzato in grado di gestire più client TCP contemporaneamente, **senza** aprire un thread per connessione.

Questo vincolo è il cuore didattico del progetto: dimostrare che l'I/O *multiplexing* (`select()` / `epoll()`) risolve lo stesso problema della concorrenza che normalmente si affronterebbe con i thread, ma restando in un singolo flusso di esecuzione.

Per questo motivo abbiamo scelto di implementare **entrambi** i meccanismi sullo stesso identico protocollo applicativo, per poterli confrontare direttamente.

Definizione del Problema

Una chat multi-utente in cui un server centrale riceve messaggi da N client e li ridistribuisce a tutti gli altri. Il server usa `select()` o `epoll()` per gestire connessioni multiple in un singolo thread, senza bloccarsi su nessun client. I client possono organizzarsi in stanze indipendenti; il broadcast avviene solo all'interno della stessa stanza.

- Comunicazione tramite protocollo TCP
- Gestione contemporanea di più connessioni, in un singolo thread
- Identificazione degli utenti tramite nickname
- Supporto a stanze indipendenti

Architettura del progetto

Suddivisione in file

```
Progetto_Laboratorio/
|
+-- include/
|   +-- chat.h          # struttura ClientInfo, costanti, prototipi
|   '-- tls.h           # contesti SSL lato server/client
|
+-- src/
|   +-- registry.c      # nickname, stanze, framing, comandi, broadcast
|   +-- main_select.c   # main loop del server basato su select()
|   +-- main_epoll.c    # main loop del server basato su epoll()
|   +-- chat_client.c   # client interattivo dedicato
|   '-- tls.c           # configurazione OpenSSL (certificato)
|
+-- build/              # file oggetto (.o), dalla compilazione
+-- bin/                # eseguibili finali, dalla compilazione
+-- Makefile
'-- README.md
```

Creazione del server TCP

Scelta di design

La logica applicativa (nickname, stanze, broadcast, framing) è isolata in `registry.c` ed è **identica** per entrambe le varianti: cambia solo il meccanismo di attesa degli eventi I/O.

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

struct sockaddr_in addr = {
    .sin_family = AF_INET,
    .sin_port = htons(PORT),
    .sin_addr.s_addr = INADDR_ANY,
};
bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
listen(server_fd, 16);
```

- `socket()` crea il descrittore del server
- `bind()` associa porta e indirizzo
- `listen()` mette la socket in ascolto, coda di backlog 16

select() e multiplexing I/O

```
fd_set master_set, read_fds;
FD_ZERO(&master_set);
FD_SET(server_fd, &master_set);
int max_fd = server_fd;

while (1) {
    read_fds = master_set;          /* select() la sovrascrive */
    select(max_fd + 1, &read_fds, NULL, NULL, NULL);

    for (int fd = 0; fd <= max_fd; fd++) {
        if (FD_ISSET(fd, &read_fds)) { /* fd pronto */ }
    }
}
```

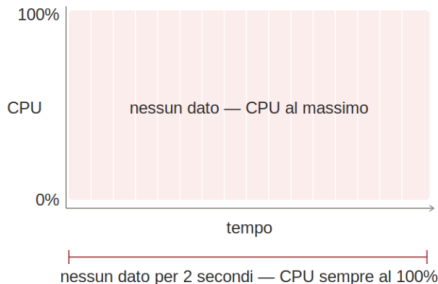
- read_fds va ricopiata da master_set ad ogni ciclo
- il controllo dei fd pronti è un ciclo lineare su 0..max_fd
- limite pratico: FD_SETSIZE (tipicamente 1024)

Busy-waiting vs I/O multiplexing

Busy-waiting — SBAGLIATO

100%
CPU sprecata

```
Loop infinito:  
while(1) {  
    recv() → EAGAIN  
    recv() → EAGAIN  
    recv() → EAGAIN  
}
```



I/O MULTIPLEXING — CORRETTO

~0%
CPU a riposo

Il kernel avvisa solo
quando ci sono dati:
zero spreco di CPU

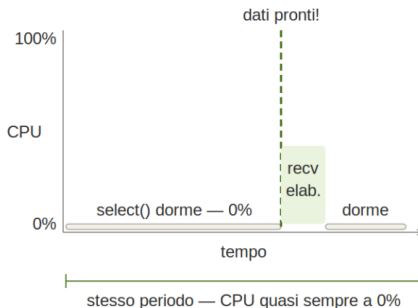


Figura: Il problema del busy-waiting

epoll(): l'alternativa moderna

```
int epfd = epoll_create1(0);
struct epoll_event ev = {.events = EPOLLIN, .data.fd = server_fd};
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

struct epoll_event events[MAX_EVENTS];
while (1) {
    int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
    for (int i = 0; i < n; i++) {
        int fd = events[i].data.fd;  /* solo i fd pronti */
    }
}
```

- il kernel mantiene l'insieme dei fd monitorati (epoll_ctl)
- epoll_wait() restituisce **solo** i fd pronti, non tutti
- nessuna ricopiatura di insiemi ad ogni ciclo

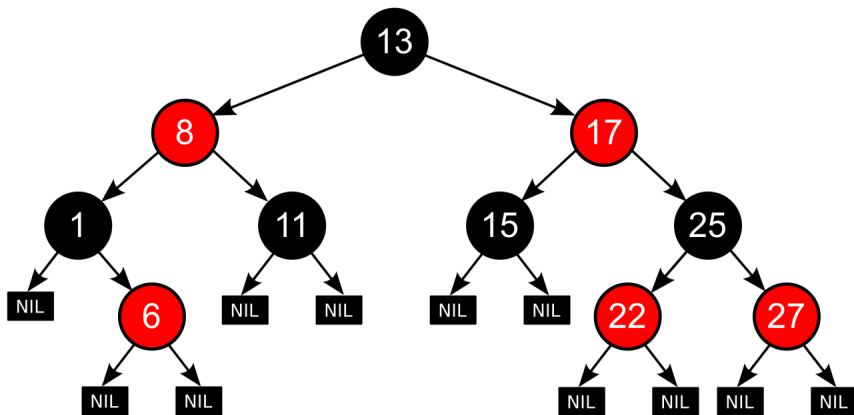


Figura: Struttura dati albero rosso-nero

select() vs epoll(): confronto

Osservazioni sperimentali

Stessa logica applicativa, stesso protocollo, stesso comportamento verificato con gli stessi test.

	select()	epoll()
Complessità per ciclo	$O(\max_fd)$	$O(\text{fd pronti})$
Stato mantenuto da	user space (ricopiato)	kernel space
Limite connessioni	FD_SETSIZE	nessun limite fisso
Portabilità	POSIX, ovunque	solo Linux
Complessità del codice	minore	leggermente maggiore

Per un progetto didattico con poche decine di client la differenza di prestazioni è trascurabile; `epoll()` diventa rilevante a partire da centinaia/migliaia di connessioni simultanee (“problema C10K”).

Struttura dati ClientInfo

```
typedef struct {
    int      fd;
    SSL      *ssl;
    char      nickname[MAX_NAME];
    char      room[MAX_ROOM];
    int      active;
    char      inbuf[MAX_LINE];    // buffer di accumulo per il framing
    size_t    inlen;
} ClientInfo;

ClientInfo clients[MAX_CLIENTS]; /* indicizzato per fd */
```

- fd, nickname, room, active: come da consegna
- inbuf/inlen: aggiunta necessaria per gestire correttamente lo streaming TCP (vedi slide successiva)
- ssl: sessione TLS associata al fd, introdotta con la cifratura del canale

Framing dei messaggi: il problema dei confini TCP

TCP è un flusso di byte, non di messaggi

Una singola `recv()` può restituire mezzo messaggio, un messaggio e mezzo, o più messaggi insieme. Trattare ogni `recv()` come un messaggio completo funziona per caso con client interattivi lenti (es. netcat riga per riga), ma non è garantito dal protocollo.

Soluzione adottata

Ogni client accumula i byte ricevuti in `inbuf` finché non trova un `'\n'`: solo a quel punto la riga viene passata alla logica applicativa. Il resto (messaggio parziale) resta in buffer in attesa dei byte successivi.

Verificato

Testato inviando un messaggio diviso artificialmente in due `send()` separate: il server lo ricompone correttamente in un'unica riga.

Comandi /nick e /join

```
if (strncmp(line, "/nick ", 6) == 0) {
    strncpy(c->nickname, line + 6, MAX_NAME - 1);
    c->nickname[MAX_NAME - 1] = '\0';
    /* ack al client */
} else if (strncmp(line, "/join ", 6) == 0) {
    strncpy(c->room, line + 6, MAX_ROOM - 1);
    c->room[MAX_ROOM - 1] = '\0';
    /* ack al client */
} else {
    broadcast_line(c, line);
}
```

- `strncmp()` riconosce il comando a inizio riga
- `line + 6` salta il prefisso ("`/nick` " o "`/join` ")
- il client riceve una conferma testuale del cambio effettuato

Broadcast filtrato per stanza

```
for (int j = 0; j < MAX_CLIENTS; j++) {  
    if (!clients[j].active) continue;  
    if (clients[j].fd == sender->fd) continue;  
    if (strcmp(clients[j].room, sender->room) != 0) continue;  
  
    SSL_write(clients[j].ssl, out, n);  
}
```

Filtri applicati

- il destinatario deve essere attivo
- il destinatario non deve essere il mittente stesso
- mittente e destinatario devono stare nella stessa stanza

SSL_write() non ha un equivalente di MSG_NOSIGNAL: SIGPIPE è ignorato globalmente all'avvio con `signal(SIGPIPE, SIG_IGN)`.

Un client dedicato: `fgets()` contro `select()`

Il problema

Il client multiplexa con `select()` tastiera e socket. La prima versione leggeva le righe con `fgets()`: se in un'unica `read()` arrivano due comandi di fila (es. `/nick` e `/join` incollati), `fgets()` li bufferizza entrambi ma ne restituisce uno solo per volta — e `select()`, che ragiona sul fd a livello kernel, non segnala più nulla finché non arrivano byte *nuovi*. Il secondo comando resta bloccato.

Soluzione: framing manuale, come lato server

Si abbandona `fgets()` in favore di `read()` grezza su `STDIN_FILENO`, con lo stesso principio di `feed_bytes()`: si accumula in un buffer proprio e si invia ogni riga completa non appena trovata, così la readiness di `select()` corrisponde sempre a ciò che viene consumato.

Cifratura del canale con TLS

Nonostante il nome, la versione iniziale del progetto non cifrava nulla. Si è integrato TLS tramite OpenSSL, sostituendo `recv()/send()` con `SSL_read()/SSL_write()` su tutti e tre gli eseguibili.

Scelta di design: handshake bloccante

L'handshake TLS richiede più round-trip e, per gestirlo in modo pienamente asincrono dentro `select()/epoll()`, servirebbe una vera macchina a stati (`SSL_ERROR_WANT_READ/WANT_WRITE` non seguono la direzione attesa dal chiamante). Si esegue quindi `SSL_accept()` in modo **bloccante**, subito dopo `accept()` e prima di aggiungere il client al set monitorato: per la durata dell'handshake il server non serve gli altri client, un costo esplicito e accettato.

SSL_read() contro select(): il bug dei ticket

Il problema

Con l'handshake funzionante, due client reali restavano muti: nessuna risposta, nemmeno al primo /nick. In TLS 1.3 il server invia di norma, non richiesti, due messaggi NewSessionTicket subito dopo l'handshake. SSL_read() li consuma internamente cercando altri byte dal socket bloccante — se in quel momento non c'è altro da leggere, resta bloccata a tempo indeterminato, e con essa l'intero ciclo eventi: il client non torna mai a select().

Soluzione

SSL_CTX_set_num_tickets(ctx, 0) elimina la causa alla radice (la ripresa di sessione non serve a questo progetto). In più, dopo ogni SSL_read() si controlla SSL_pending() e si continua a leggere finché promette dati già decifrati: stesso principio di feed_bytes(), un livello più in basso.

Test funzionali eseguiti

- Connessione di un singolo client
- Connessione simultanea di più client
- Cambio nickname con `/nick`
- Cambio stanza con `/join`
- Broadcast ricevuto solo dagli utenti della stessa stanza
- Nessun eco del messaggio al mittente
- Messaggio applicativo spezzato su due `send()` distinte, ricomposto correttamente — ripetuto sul canale cifrato
- Gestione della disconnessione (`recv()/SSL_read() ≤ 0`)
- Stesso comportamento verificato su `chat_select` e `chat_epoll`
- Test ripetuti sia con `netcat` sia con `chat_client`
- Connessione in chiaro con `netcat` verso il server TLS, correttamente respinta

Esempio di sessione

```
massimilianosparta@massimilianosparta-RLEF-XX: ~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ make
rm -rf build bin
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ make
gcc -Wall -Wextra -O2 -Iinclude -c src/main_select.c -o build/main_select.o
gcc -Wall -Wextra -O2 -Iinclude -c src/registry.c -o build/registry.o
gcc -Wall -Wextra -O2 -Iinclude -c src/tls.c -o build/tls.o
gcc build/main_select.o build/registry.o build/tls.o -o bin/chat_select -lssl -lcrypto
gcc -Wall -Wextra -O2 -Iinclude -c src/main_epoll.c -o build/main_epoll.o
gcc build/main_epoll.o build/registry.o build/tls.o -o bin/chat_epoll -lssl -lcrypto
gcc -Wall -Wextra -O2 -Iinclude -c src/chat_client.c -o build/chat_client.o
gcc build/chat_client.o build/tls.o -o bin/chat_client -lssl -lcrypto
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./chat_epoll
bash: ./chat_epoll: No such file or directory
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./bin/chat_epoll
=== SecureChat (epoll) su porta 8080 ===
[+] client fd=5 connesso (TLS) da 127.0.0.1:35836
[+] client fd=6 connesso (TLS) da 127.0.0.1:33684
[+] client fd=7 connesso (TLS) da 127.0.0.1:58148
[]

massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./bin/chat_client
Connesso (TLS, TLSv1.3) a SecureChat su 127.0.0.1:8080
Comandi: /nick <nome> /join <stanza> Ctrl-D per uscire
/nick max
=> ora sei conosciuto come max
/join tennis
>> sei entrato nella stanza tennis
ciao
>> [tennis][Sino] ciao
Buonasera
>> [tennis][Ugo] Ciao
[]

massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat
massimilianosparta@massimilianosparta-RLEF-XX:~/SecureChat$ ./bin/chat_client
Connesso (TLS, TLSv1.3) a SecureChat su 127.0.0.1:8080
Comandi: /nick <nome> /join <stanza> Ctrl-D per uscire
/nick Ugo
=> ora sei conosciuto come Ugo
Ciao
/join tennis
>> sei entrato nella stanza tennis
Ciao
[]
```

Discussione dei risultati

- Condividere la logica applicativa tra `select()` ed `epoll()` ha permesso un confronto diretto e onesto: stesso protocollo, stesso comportamento osservabile.
- Il problema più insidioso non è stato il multiplexing in sé, ma il framing, in tre manifestazioni via via meno visibili: nel protocollo (`feed_bytes()`), nel client (`fgets()` contro `select()`), e dentro OpenSSL (`SSL_read()` contro `select()`).
- In tutti e tre i casi il sintomo non è un crash, solo un silenzio: qualunque libreria interponga un proprio buffer tra il fd e il chiamante può rompere l'assunzione che "niente di pronto" equivalga a "niente da elaborare".
- L'array `clients[MAX_CLIENTS]` indicizzato per fd è $O(1)$ ma spreca memoria se i fd sono sparsi; per un progetto di queste dimensioni è un compromesso accettabile.
- Restare in un singolo thread, come richiesto dalla consegna, ha semplificato la gestione dello stato condiviso: nessun mutex, nessuna race condition da temere sulla struttura `clients`.

Risultato raggiunto

SecureChat è una chat multiutente TCP funzionante, con due implementazioni equivalenti del multiplexing I/O (`select()` ed `epoll()`), stanze, nickname, un framing dei messaggi robusto rispetto ai confini imposti da TCP, un client dedicato per i test end-to-end e un canale cifrato con TLS.

- confermata la validità dell'I/O multiplexing come alternativa al thread-per-client
- quantificate le differenze pratiche tra `select()` ed `epoll()`
- risolto un problema di framing spesso trascurato nelle implementazioni didattiche, a tre livelli diversi: protocollo, client, TLS
- cifrato il canale con TLS, con un bilancio onesto di cosa questo garantisce (riservatezza e integrità) e cosa no (autenticità del server, senza una CA reale; autenticazione degli utenti)

Riferimenti bibliografici

-  RFC 9293 - Transmission Control Protocol (TCP)
-  RFC 793 - Transmission Control Protocol (TCP) legacy
-  Linux man-pages: `select(2)`, `epoll(7)`, `socket(2)`, `accept(2)`
-  W. Richard Stevens, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3^a ed., Addison-Wesley, 2003.
-  T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, Cap. 13 (Red-Black Trees)

Grazie per l'attenzione!

Informazioni

Autori:	Massimiliano Spartà e Simone Adamo
Anno Accademico:	2025/2026
Corso:	Laboratorio di Reti e Sistemi Distribuiti (L-31)
Docente:	R. Marino
Fonti principali:	RFC 9293, Linux man-pages